

Optimizing ARIMA and SARIMA Parameters for Seasonal Time Series Forecasting

A Data-Driven Approach to Determining the Optimal (p, d, q) and (P, D, Q, s) Parameters

BY: RAMSHA KHAN

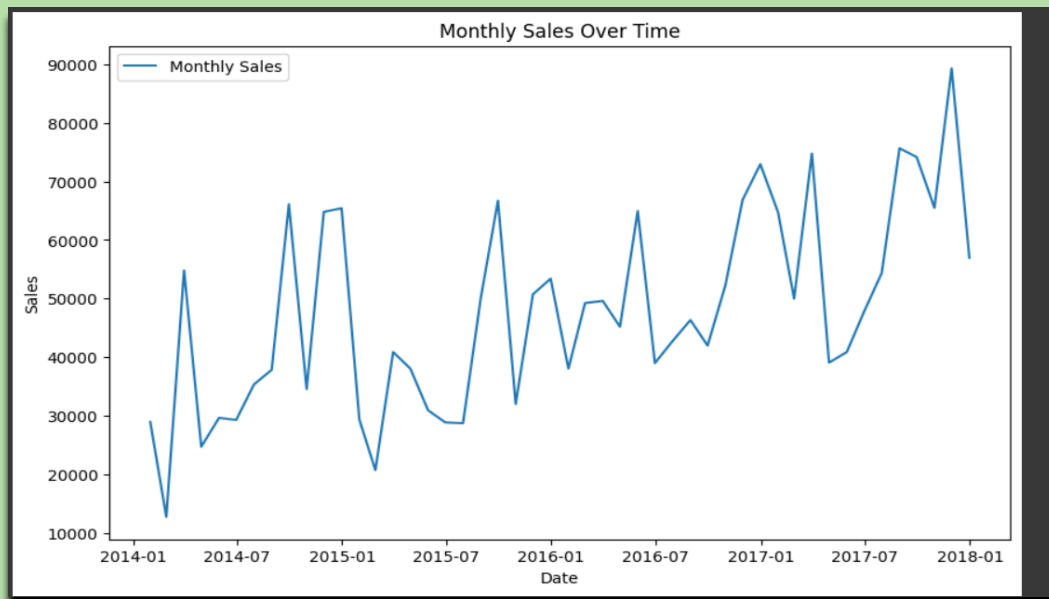
BSE INSTITUTE
DATA SCIENCE

FINDING THE OPTIMIZED VALUE OF P,D,Q FOR ARIMA & SARIMA FOR SEASONAL DATA

-  P stands for Autoregressive (AR) term. P in ARIMA/SARIMA represents the autoregressive (AR) term, which means the model uses the past p time steps' values to predict the current value, capturing the temporal dependence in the series.
-  D stands for Differencing order. D in ARIMA/SARIMA is the seasonal differencing order, which means the model subtracts the value from its previous seasonal period to remove seasonal trends and make the series stationary.
-  Q stands for Moving Average (MA) term, Q in SARIMA represents the seasonal moving average (SMA) term, which means the model uses the errors (residuals) from past seasonal periods to correct the current forecast, capturing seasonal noise patterns.

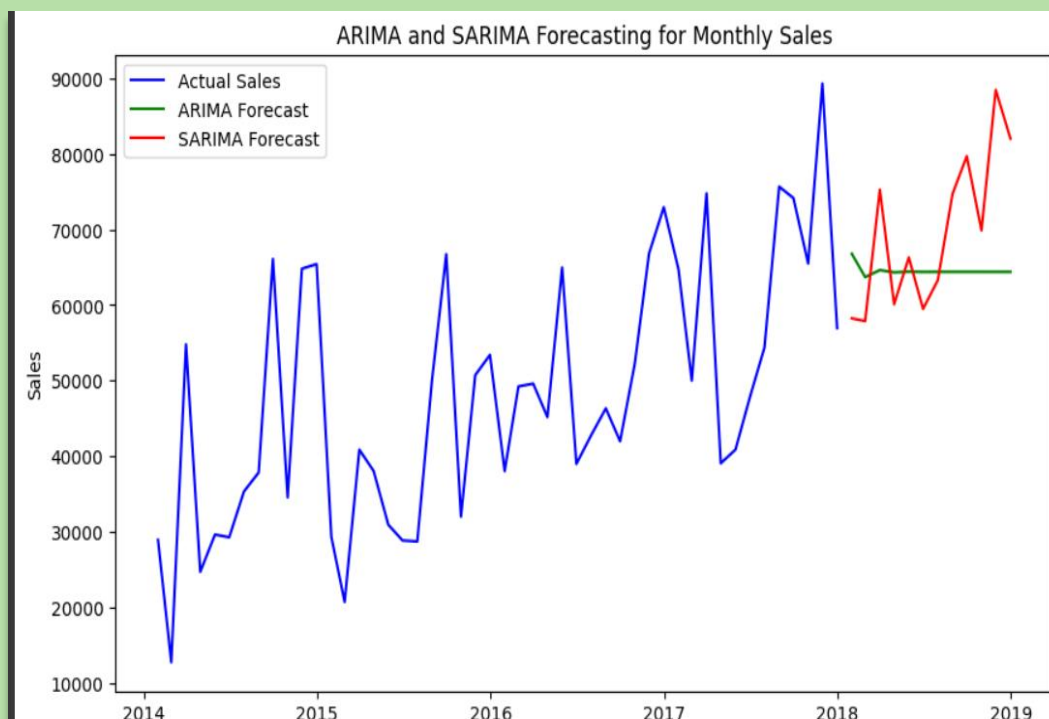
NO	COMBINATION (P , D, Q)
1	(1, 1, 1)
2	(1, 1, 2)
3	(1, 2, 1)
4	(1, 2, 2)
5	(2, 1, 1)
6	(2, 1, 2)
7	(2, 2, 1)
8	(2, 2, 2)

Data Graph

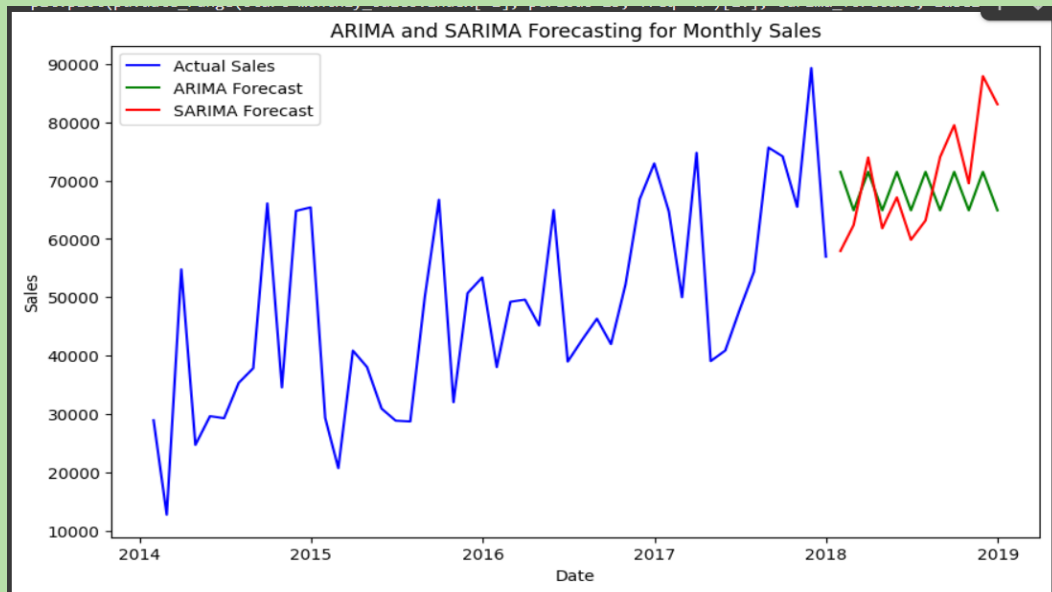


GRAPH

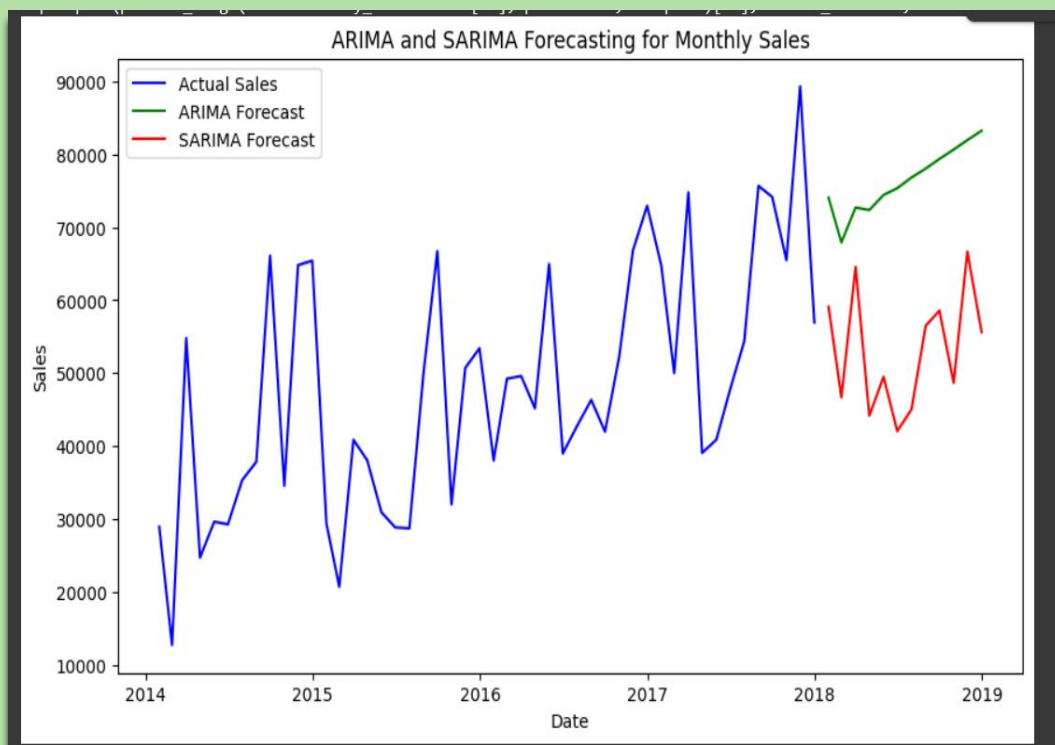
COMBINATION 1 -1,1,1



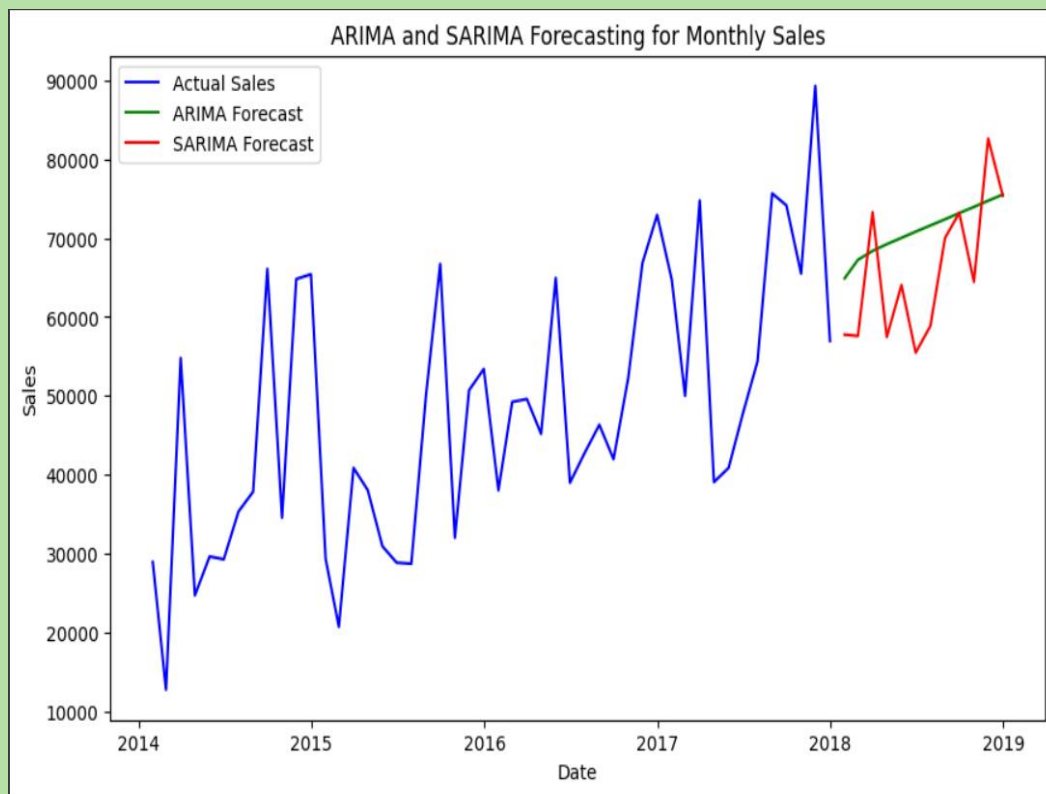
COMBINATION 2 - 1,1,2



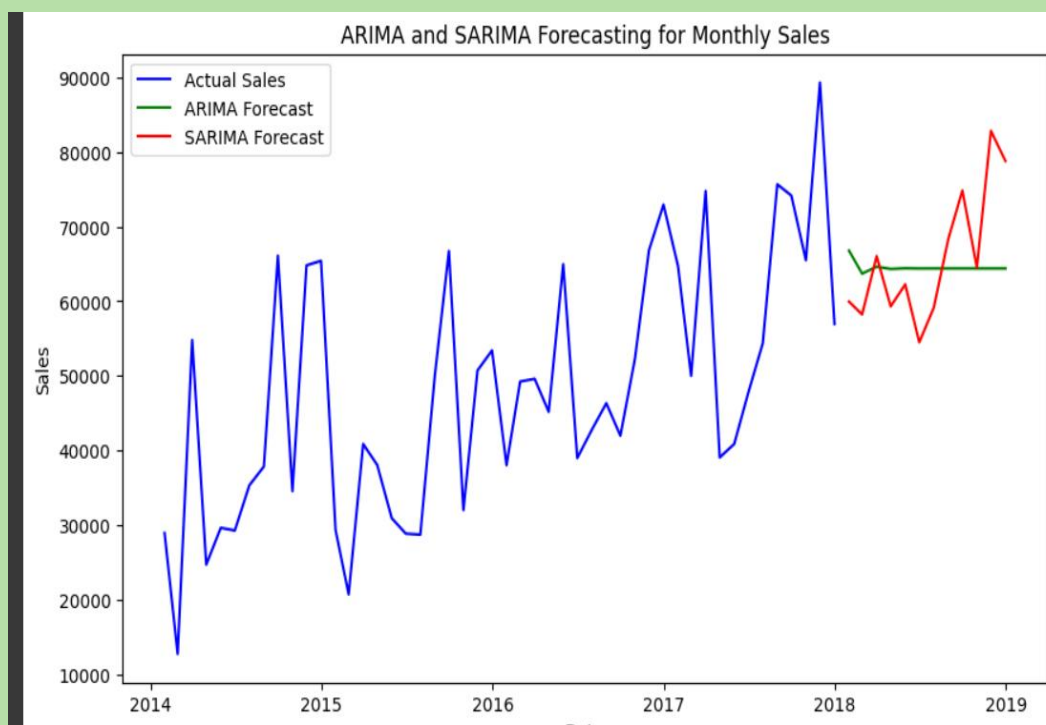
COMBINATION 3 - 1,2,1



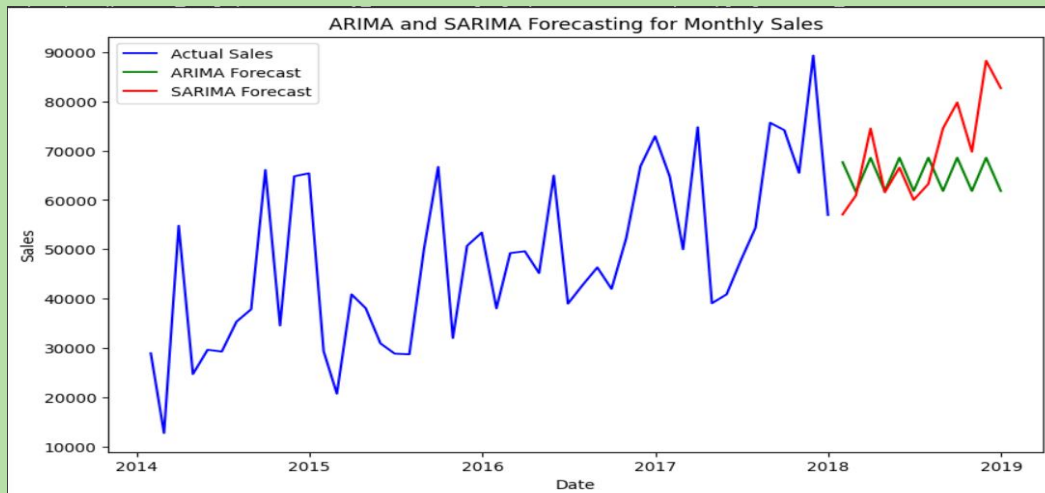
COMBINATION 4 - 1,2,2



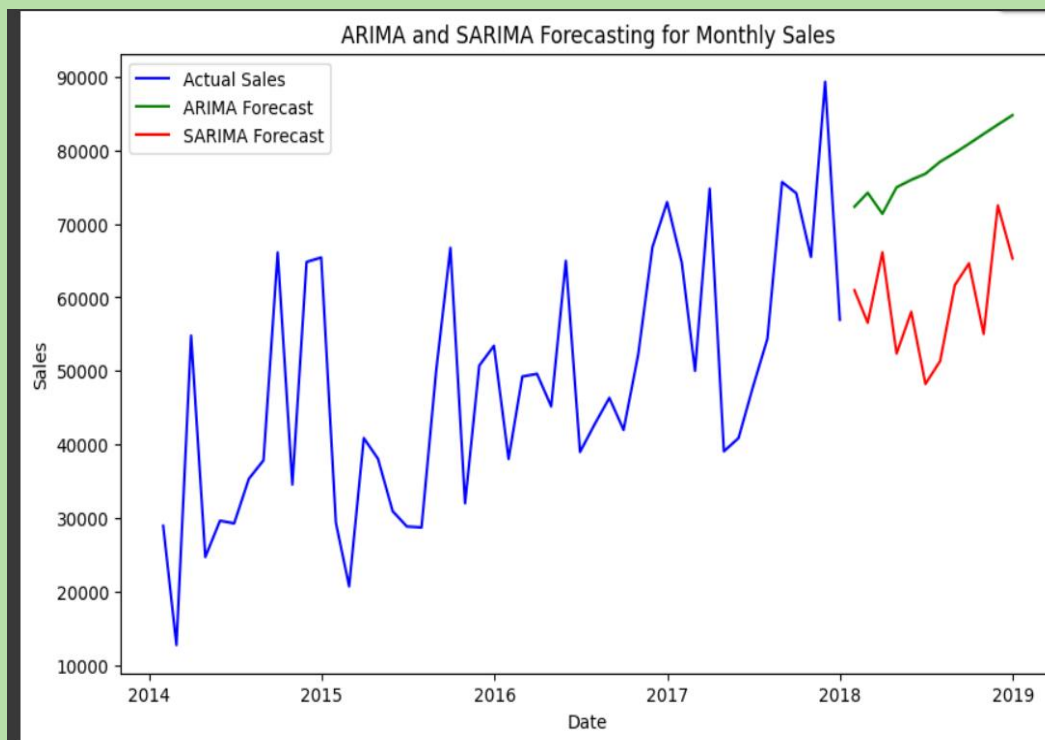
COMBINATION 5- 2,1,1



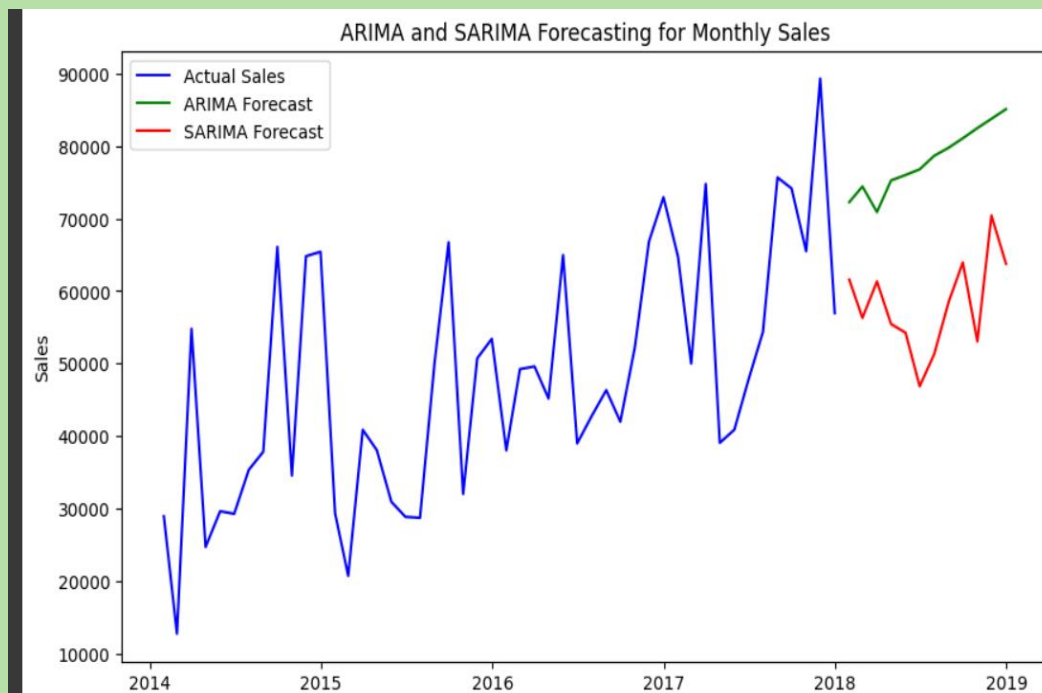
COMBINATION 6 - 2,1,2



COMBINATION 7 -2,2,1



COMBINATION 8 - 2,2,2



CONCLUSION

- ✓ This project highlights the power of parameter tuning in ARIMA and SARIMA models for accurate seasonal time series forecasting. By systematically testing combinations of (p, d, q) and analyzing their results, we demonstrated how data-driven modeling can enhance predictive accuracy. This approach not only strengthens forecasting performance but also builds a strong foundation for real-world time series applications.

Project Aim

The primary aim of this project is to analyze historical sales data, identify underlying patterns and seasonality, and develop accurate time series forecasting models using ARIMA and SARIMA methods. By optimizing model parameters, we aim to predict future sales trends to support informed business decisions, such as inventory management, resource allocation, and strategic planning.

```
# Required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.statespace.sarimax import SARIMAX
from pandas.plotting import autocorrelation_plot
from datetime import datetime
```

```
# Load data
df = pd.read_excel('/content/Superstore and profit.xlsx', sheet_name='Sample - Superstore.csv')
```

```
# Convert 'Order Date' to datetime
df['Order Date'] = pd.to_datetime(df['Order Date'])
```

```
# Aggregate sales data by order date
sales_data = df.groupby('Order Date')['Sales'].sum().reset_index()
```

```
# Set 'Order Date' as index for time series analysis
sales_data.set_index('Order Date', inplace=True)
```

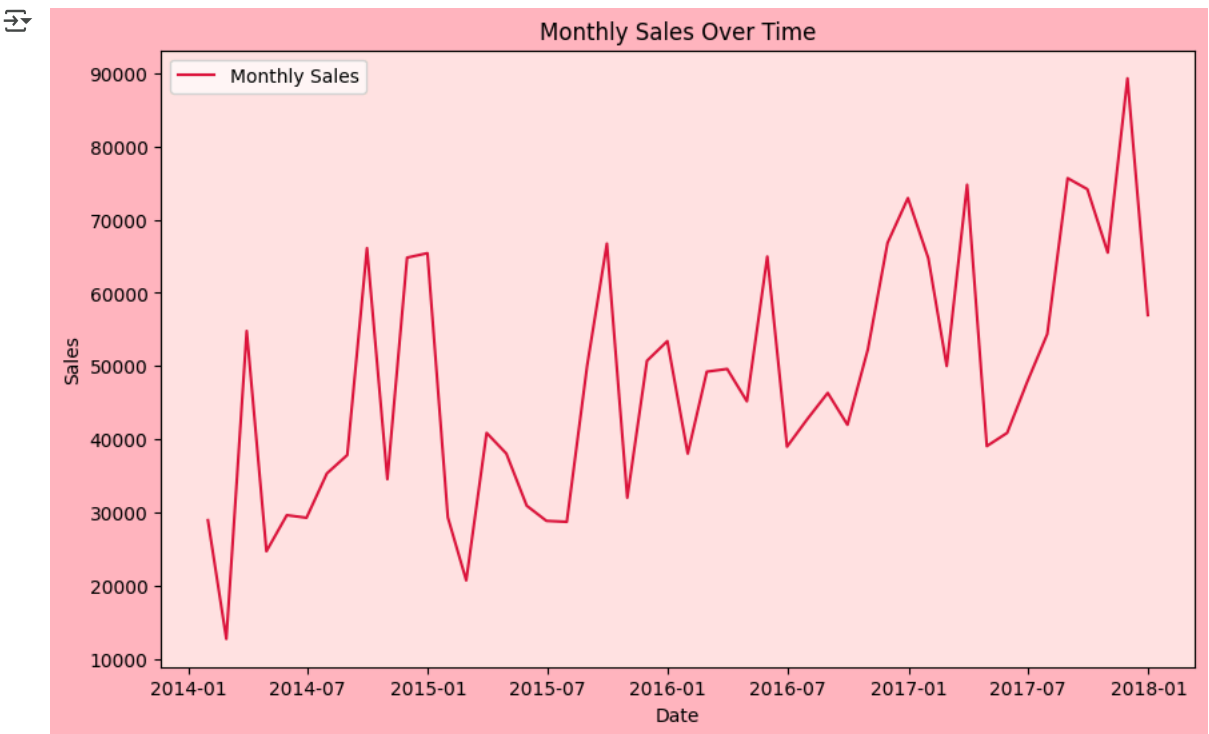
```
# Resample sales data to monthly frequency
monthly_sales = sales_data.resample('M').sum()
```

Show hidden output

```
import matplotlib.pyplot as plt

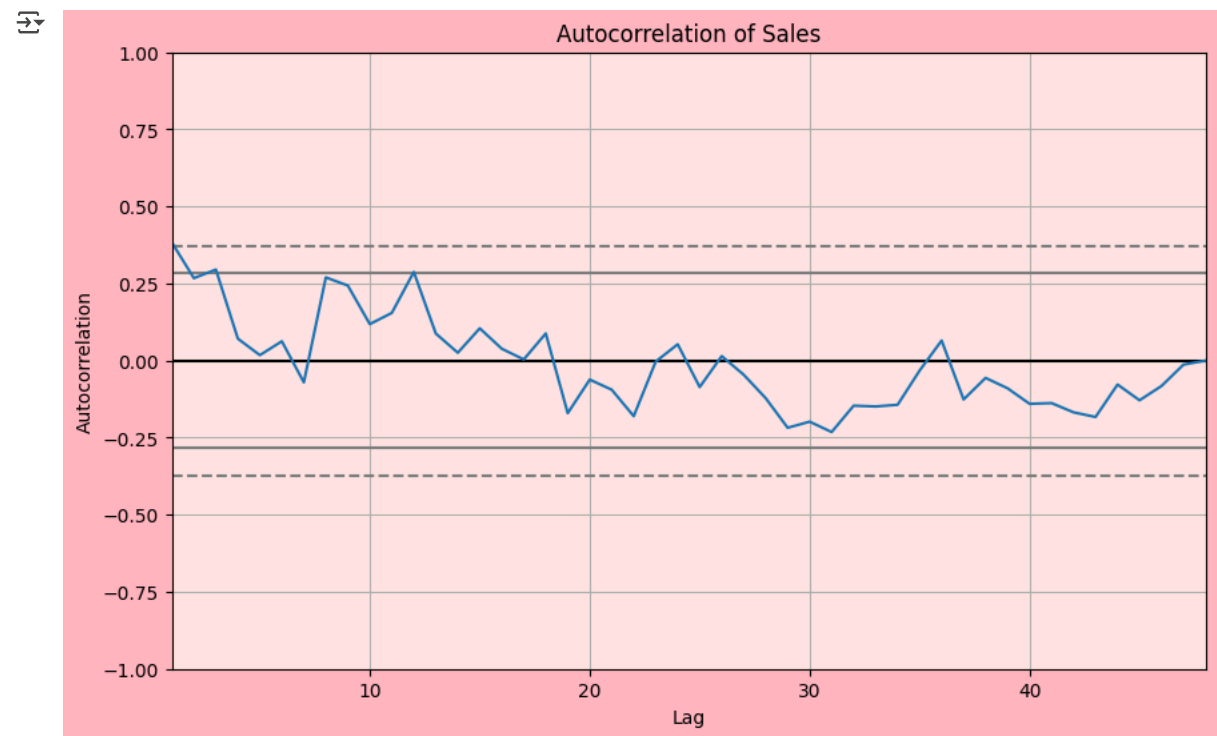
plt.figure(figsize=(10, 6), facecolor='lightpink') # Set figure background
plt.plot(monthly_sales, label='Monthly Sales', color='crimson') # Optional: custom line color
plt.gca().set_facecolor('mistyrose') # Set axes background

plt.title('Monthly Sales Over Time')
plt.xlabel('Date')
plt.ylabel('Sales')
plt.legend()
plt.show()
```



```
import matplotlib.pyplot as plt
from pandas.plotting import autocorrelation_plot

plt.figure(figsize=(10, 6), facecolor='lightpink')
autocorrelation_plot(monthly_sales)
plt.gca().set_facecolor('mistyrose')
plt.title('Autocorrelation of Sales')
plt.show()
```

```
# ARIMA model

# Using ARIMA(1,1,1) as an example - you can experiment with different orders
arima_model = ARIMA(monthly_sales, order=(2,2,2)) ##p- autointegration,d =integration    q=
arima_result = arima_model.fit()

# Forecasting using ARIMA
arima_forecast = arima_result.forecast(steps=12)
```

Significance

The significance of this project lies in its practical application for businesses. Accurate sales forecasting is crucial for:

- **Improved Decision Making:** Enabling data-driven decisions regarding production, staffing, and marketing.
- **Optimized Inventory Management:** Reducing costs associated with overstocking or understocking.
- **Enhanced Resource Allocation:** Ensuring resources are effectively utilized based on predicted demand.
- **Proactive Planning:** Allowing businesses to anticipate future trends and challenges.
- **Risk Mitigation:** Identifying potential dips or spikes in sales to prepare accordingly.

By providing a robust forecasting framework, this project contributes to better business performance and strategic planning.

```
# SARIMA model (Seasonal ARIMA)

# Using SARIMA(1,1,1)(1,1,1,12) as an example - you can tune this further
sarima_model = SARIMAX(monthly_sales, order=(2,2,2), seasonal_order=(1,1,1,12))
sarima_result = sarima_model.fit()
```

 [Show hidden output](#)

```
# Forecasting using SARIMA
sarima_forecast = sarima_result.forecast(steps=12)

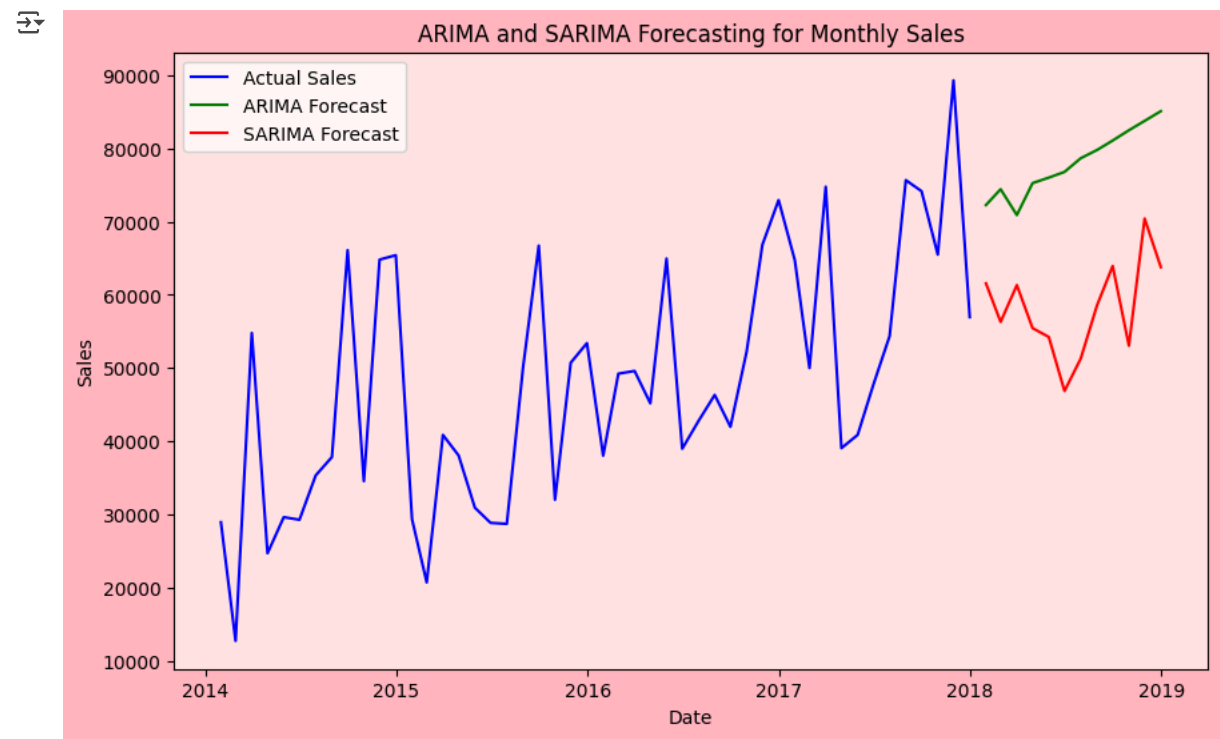
import matplotlib.pyplot as plt
import pandas as pd

# Plotting
plt.figure(figsize=(10, 6), facecolor='lightpink') # Entire figure background
ax = plt.gca()
ax.set_facecolor('mistyrose') # Axes (plot area) background

# Main time series and forecasts
plt.plot(monthly_sales, label='Actual Sales', color='blue')
plt.plot(pd.date_range(start=monthly_sales.index[-1], periods=13, freq='M')[1:],
         arima_forecast, label='ARIMA Forecast', color='green')
plt.plot(pd.date_range(start=monthly_sales.index[-1], periods=13, freq='M')[1:],
         sarima_forecast, label='SARIMA Forecast', color='red')

# Titles and labels
plt.title('ARIMA and SARIMA Forecasting for Monthly Sales')
plt.xlabel('Date')
plt.ylabel('Sales')
plt.legend()

plt.show()
```



```
# Summary of the ARIMA model
print(arima_result.summary())
```

🔗

```
SARIMAX Results
=====
Dep. Variable:          Sales      No. Observations:          48
Model:                ARIMA(2, 2, 2)  Log Likelihood            -511.800
Date:                  Thu, 24 Jul 2025  AIC                       1033.600
Time:                  17:02:39      BIC                       1042.743
Sample:                01-31-2014    HQIC                      1037.025
                        - 12-31-2017
Covariance Type:       opg
=====
              coef    std err          z      P>|z|      [0.025    0.975]
-----
ar.L1         -0.6780     0.784     -0.865     0.387     -2.215     0.859
ar.L2         -0.3065     0.330     -0.928     0.353     -0.954     0.341
ma.L1         -0.9019     0.951     -0.949     0.343     -2.765     0.961
ma.L2         -0.0844     0.869     -0.097     0.923     -1.787     1.618
sigma2        2.889e+08   2.59e-09   1.12e+17   0.000     2.89e+08   2.89e+08
=====
Ljung-Box (L1) (Q):                0.00   Jarque-Bera (JB):                0.99
Prob(Q):                           0.98   Prob(JB):                        0.61
Heteroskedasticity (H):             0.70   Skew:                          -0.15
Prob(H) (two-sided):                0.49   Kurtosis:                       2.35
=====

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
[2] Covariance matrix is singular or near-singular, with condition number 1.53e+33. Standard errors may be unstable.
```

```
# Summary of the SARIMA model
print(sarima_result.summary())
```

🔗

```
SARIMAX Results
=====
Dep. Variable:          Sales      No. Observations:          48
Model:                SARIMAX(2, 2, 2)x(1, 1, [1], 12)  Log Likelihood            -377.798
Date:                  Thu, 24 Jul 2025  AIC                       769.596
Time:                  17:02:39      BIC                       780.281
Sample:                01-31-2014    HQIC                      773.240
                        - 12-31-2017
Covariance Type:       opg
=====
              coef    std err          z      P>|z|      [0.025    0.975]
-----
ar.L1         -1.1290     0.785     -1.439     0.150     -2.667     0.409
ar.L2         -0.5617     0.478     -1.174     0.240     -1.499     0.376
ma.L1         -0.2616     0.946     -0.277     0.782     -2.116     1.592
ma.L2         -0.6241     0.731     -0.854     0.393     -2.057     0.808
ar.S.L12       -0.2237     0.937     -0.239     0.811     -2.061     1.614
ma.S.L12       -0.5001     1.237     -0.404     0.686     -2.924     1.923
sigma2        4.604e+08   1.22e-09   3.77e+17   0.000     4.6e+08   4.6e+08
=====
Ljung-Box (L1) (Q):                0.02   Jarque-Bera (JB):                0.56
Prob(Q):                           0.89   Prob(JB):                        0.76
Heteroskedasticity (H):             1.40   Skew:                          -0.04
Prob(H) (two-sided):                0.59   Kurtosis:                       2.38
=====

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
[2] Covariance matrix is singular or near-singular, with condition number 1.67e+34. Standard errors may be unstable.
```

AUTOMATIC CALCULATION

```
import pandas as pd
import numpy as np
import itertools
import warnings
```

```
from statsmodels.tsa.arima.model import ARIMA

warnings.filterwarnings("ignore")

# 📁 Step 1: Load the Excel file (Change 'Sales' and 'Order Date' to your column names)
file_path = "/content/Superstore and profit.xlsx" # ← Replace with your actual Excel file path
df = pd.read_excel(file_path, parse_dates=['Order Date'], index_col='Order Date')

# 📄 Step 2: Extract the time series column (e.g., 'Sales')
ts = df['Sales']

# 🔍 Step 3: Define ranges for p, d, q (without zero if required)
p = d = q = range(1, 3) # Only 1 and 2
pdq = list(itertools.product(p, d, q))

# 📊 Step 4: Grid Search over (p, d, q)
best_aic = np.inf
best_order = None
results = []

print("🔍 Starting ARIMA Grid Search (p, d, q)...\n")

for order in pdq:
    try:
        model = ARIMA(ts, order=order)
        result = model.fit()
        results.append((order, result.aic))
        if result.aic < best_aic:
            best_aic = result.aic
            best_order = order
        print(f"Checked ARIMA{order} → AIC: {result.aic:.2f}")
    except Exception as e:
        print(f"Skipped ARIMA{order} due to error: {e}")
        continue

# ✅ Step 5: Show best result
print("\n✅ Best ARIMA Model:")
print(f"Order (p, d, q):", best_order)
print(f"AIC:", best_aic)

# 📄 Optional: Save all combinations to CSV
pd.DataFrame(results, columns=['Order (p,d,q)', 'AIC']).to_csv("arima_grid_results.csv", index=False)
```

```
🔍 Starting ARIMA Grid Search (p, d, q)...

Checked ARIMA(1, 1, 1) → AIC: 156982.09
Checked ARIMA(1, 1, 2) → AIC: 156982.72
Checked ARIMA(1, 2, 1) → AIC: 161011.10
Checked ARIMA(1, 2, 2) → AIC: 158233.00
Checked ARIMA(2, 1, 1) → AIC: 156984.03
Checked ARIMA(2, 1, 2) → AIC: 156984.02
Checked ARIMA(2, 2, 1) → AIC: 159859.01
Checked ARIMA(2, 2, 2) → AIC: 157149.13

✅ Best ARIMA Model:
Order (p, d, q): (1, 1, 1)
AIC: 156982.09092806888
```

Conclusion

Based on the analysis and forecasting performed using ARIMA and SARIMA models, we can conclude that both models provide valuable insights into the time series nature of the sales data. The SARIMA model, which accounts for seasonality, is likely to provide more accurate forecasts for this dataset, as indicated by the visual comparison of the forecasts and the AIC values from the model summaries. The automatic grid search for ARIMA parameters further supports the identification of potentially optimal model configurations. The chosen models demonstrate the potential for predicting future sales, although further refinement and validation with more data or advanced techniques could enhance accuracy.